

lakearch — The Data Model

RULESET. COMPLETE, DOMAIN-FREE, TECHNOLOGY-FREE.

Authoritative English version; the German translation follows below.

Preamble. This document describes the *lakearch* data model exhaustively and in general. It names no domain, no application, no technology. It is framed as an axiom system: few primitive notions, and from them unambiguously following rules. Every rule holds without exception; where an exception would seem necessary, the rule is wrongly framed.

Notation. A, B, K denote arbitrary data; they are abstract symbols, for lakearch knows no contents (§2.1, §1.4). " $A \triangleright K$ " reads "A owns the context K".

§1 Scope

1.1 lakearch **stores** data and their contexts.

1.2 lakearch **traverses** along contexts — forward as well as backward.

1.3 lakearch **matches structurally**: equality at the content/address level, "does a context point to a datum?", membership in a set given by a context.

1.4 lakearch **does not interpret** (contents remain opaque, §2.1; meaning arises only through the vocabulary of the derivation, §14), **does not compute** (no arithmetic, ordering, aggregation), **does not judge** (produces no confidence, decides no identity, validates no input) and **does not sort**. lakearch is a **passive store**: it holds data without executing any logic of its own over them.

1.5 All computing, judging and sorting lies in a **layer above lakearch**. It reads by traversal and writes its results back as data.

1.6 **Cycles are permitted**. The graph is not a DAG; a context may indirectly point back to its own owner datum. Structural termination does not follow from the form but from the kind of traversal (§1.7).

1.7 Traversal occurs in two kinds:

(a) **Mechanical traversal** — the access gate (§11) and invalidation (§10) — is **deterministic, bounded** (over a visited set) **and thus cycle-safe**. It only matches; it does not compute or judge.

(b) **Exploratory traversal** — search, placement, deliberate backtracking — lies in the layer above (§1.5). It may use cycles and is not bound to determinism.

§2 The Entity

2.1 There is exactly **one** kind of entity: the **datum**. Everything that exists is a datum.

2.2 There is no second entity class — no schema, no metadata, version, time or type objects as a kind of their own. This reduction is the core principle.

2.3 A **store** is a coherent set of data under exactly one access gate (§11) — one lakearch instance. All following rules hold, at first, *within* a store; several connected stores are treated by §12. A **region** (§11), by contrast, is a *logical* separation *within* a store: **store $\supset$ region**.

§3 The Context

3.1 A **context** is not an entity of its own but a **role**: a datum K that is owned by a datum A. K itself is an ordinary datum.

3.2 Ownership is **directed**: "A $\triangleright$ K" is not symmetric.

3.3 A **reference** from A to B is expressed by A owning a context that targets B. The kind of the reference (the relation) is itself a datum.

3.4 A context may itself own contexts. **Reification is free**: statements about statements are native.

3.5 Every statement of the system is a datum owned by a datum. No other form of statement exists.

3.6 A reference always points **to an existing datum**. A target that has not yet arrived is represented by a **placeholder datum** — an ordinary datum with a context marking it as *unresolved/expected*. Later intake replaces it (§6.3) or connects via the correlation path (§5.7 b). Thus every reference stays closed: there are no dangling references, only data explicitly marked as unresolved.

§4 Type

4.1 **Type** is a special context: a datum carries its type as a context that points to a type datum.

4.2 Since a type is a datum, there is **no meta-type regress**; type statements are ordinary data.

4.3 Data have no prescribed structure. Structure arises solely through contexts. Every domain is representable **without extending the model**.

§5 Identity

5.1 Identity is **not an intrinsic property** but a statement about data — and thus itself a context.

5.2 **Storage identity**: every datum ever written carries an opaque ID, preferably a **content hash**. It addresses; it does not judge.

5.3 **Value identity**: content-equal data are the same datum — content addressing deduplicates automatically. A primitive datum exists exactly once.

5.4 **Referential identity** ("the same thing?") is expressed exclusively through contexts, never through destructive operations.

5.5 Referential identity is **graded**: a family of identity contexts of differing strength. Each carries its own contexts: affected attributes, confidence, originator, time.

5.6 Properties of a source (stable keys, known deviations, degree of trust) are contexts on the datum that represents the source.

5.7 The intake of a datum follows exactly one of three paths, **recognizable from contexts alone**:

(a) **Succession** — same source, stable key, no contradiction: a new state, the old one marked as superseded.

(b) **Correlation** — different source or no unambiguous key: always an independent datum, plus graded identity contexts.

(c) **Conflict** — no direction holds or active contradictions exist: modeled as such; resolution deferred to the read side (§8).

5.8 Which path applies is **chosen by the writing layer (§7)** — not by lakearch. lakearch *matches* the preconditions (does a stable key exist? does a contradiction context exist?, §1.3); the *choice* and any judgment are made by the layer above (§1.4).

§6 Time

6.1 Time is data. Points in time and intervals are data; time statements are special contexts.

6.2 There are **two time axes**: **recording time** (when the system learned it) and **validity time** (when the matter holds in the world). They may diverge.

6.3 The model is **append-only**: new knowledge is added as new data. A **supersession context** marks older data as outdated, without deleting it.

6.4 A "version" is nothing stored but a **read rule** (§8).

§7 Writing

7.1 The **only mutation** is **append**: exactly one new datum together with its contexts is added. It is **never changed and never deleted**. Supersession (§6.3) and the active marker (§13) are the *only* forms of "change" — both themselves merely further writes.

7.2 **What** is written and **where** it binds by context is decided by the **layer above lakearch** (§1.5): it traverses to orient (§1.7 b), matches to check (§1.3), and then writes **once**. lakearch *does not find the place and does not judge the connection*; it takes up the finished result.

7.3 **Append does not compute or judge** (§1.4). The decision criteria of the writing layer, too, lie as data in the store; to *read* them is traversal, to *apply* them is the business of the layer above. From this follows the guiding principle: *all state lies in lakearch, all computation above it — including the computation that decides where new state goes*.

7.4 If a write concerns several data, its result takes effect only once the active marker releases it (§13); until then the parts count as inactive.

§8 Resolution on Read

8.1 What graded identity (§5), time (§6) and conflicts mean is decided **on read**, not on write.

8.2 A query chooses implicitly: confidence thresholds, source weighting, point in time and time axis.

8.3 The same question to the same store may yield different results depending on the choice. Ambiguity is treated honestly on read, not hidden on write.

8.4 Reading creates nothing in the store; it **projects** from the existing store. Historical states are thus reconstructed.

§9 Structures of Identity Resolution

lakearch holds the structures; the resolving itself (thresholds, decisions) happens in the layer above (§1.5).

9.1 **Representative system**: if several data bear the same referential identity, a separate **anchor datum** exists (the class). The representatives point by membership context to the anchor.

9.2 Other things point to the **anchor**, not to a single representative. A representative is **never** converted into the anchor (§6.3).

9.3 Membership is **graded and revisable** — a context, not a destructive merge.

9.4 A **split** arises through new contexts that supersede old ones; affected representatives are re-pointed.

9.5 **Curation** adds exclusively reversible contexts (*hide, supersede, membership*); the read side filters them. Nothing is ever deleted; physical removal is compaction (§15).

§10 Materialization

10.1 A computed result may be stored as a datum; a newer one supersedes the older (§6.3).

10.2 A computed datum carries its **provenance as context** — the binding to the inputs from which it arose.

10.3 If an input changes (also retroactively, §6), the affected results are found by **backward traversal** of the provenance contexts (invalidation; mechanical, §1.7 a). Recomputation lies outside (§1.5).

§11 Access

11.1 **Description.** A **region** is a datum; membership is a context (a datum may belong to several regions). A region separates *logically within* a store (§2.3). A **permission** is a datum with contexts *subject, region, right, time, originator* — auditable and bitemporal.

11.2 **Enforcement.** Every read passes an **unbypassable gate**. The gate forms visibility from the permissions (pure structural matching: *region* $\in$ *granted regions*, §1.3; mechanical, §1.7 a) and applies it **before any resolution**.

11.3 **Filtering before resolving** is mandatory: non-visible data are removed before referential identity (§9) is resolved — so that nothing non-visible flows into a permitted result.

11.4 **Revocation** is a new permission context that supersedes the old one; future reads filter it, what has already been read remains (§6.3).

11.5 The gate is part of lakearch — *always invoked, tamper-proof, minimal*. It only matches (§1.3); it does not compute.

§12 Federation

12.1 A single store **and** several connected stores are both possible — with the same rules.

12.2 **Within a store**, regions (§11) separate logically.

12.3 **Across stores**, content-equal data bear the same content hash (§5.2) and are thereby identical across stores — automatic reconciliation. Referential identity is connected via the correlation path (§5.7 b); to take one store into another *is* this path.

12.4 Content IDs are store-independent; **anchor IDs (§9.1) are store-local** and are reconciled on merge via graded identity.

§13 Atomicity

13.1 Every data access — **reading as well as writing** — is **atomic**: indivisible and without observable intermediate state. No access ever observes a half-completed write; it sees the store either wholly before or wholly after.

13.2 For the **single write**, atomicity is structurally given: the only mutation is *append* (§7.1), and a datum is never changed and never deleted. The new datum together with its contexts becomes visible as a whole or not at all; a partially written datum is unobservable.

13.3 For the **single read**, atomicity follows from immutability: reading projects (§8.4) from already-existing, immutable data and observes no intermediate state produced by a concurrent write.

13.4 For **reading across several data** — traversal, query — atomicity follows from the implicit **time cut** (§8.2) together with append-only immutability (§7.1): the read projects (§8.4) against exactly one point in time. Since writing is solely *append* and never changes what exists, the store only grows; a concurrently appended datum lies behind the cut and stays unobserved. Thus even the composite read sees the store wholly before or wholly after a concurrent write (§13.1) — never a mixed state.

13.5 For a **restructuring concerning several data** (merging/splitting §9, permission change §11), shared visibility arises through a single concluding **active write**. Until the active marker is set, the parts count as

inactive; traversal ignores them (§7.4).

13.6 A half-completed restructuring is thereby invisible until it is complete — **atomicity without transaction machinery**. Concurrency beyond this pattern is a matter of implementation (§15).

§14 Derivation & Separation

14.1 A **domain-specific derivation** does not extend the model. It is data *within* a lakearch store: vocabulary (type and relation data), conventions, configuration.

14.2 **Separation rule**. Vocabulary belongs in the derivation. A new storage, traversal or match primitive belongs in lakearch and must be domain-free. Computing, judging and sorting belong in the layer above (§1).

14.3 lakearch knows **no** derivation by name. It provides only primitives in which every derivation expresses itself fully.

§15 Deliberately Open (Implementation & Operation)

These points are not a model component but implementation decisions:

- **ID scheme** (content hash / UUID / hybrid), **serialization, query and traversal language**.
- **Compaction**: when outdated or hidden data may be *physically* removed (tension append-only ↔ deletion duties).
- **Indexing & performance** of region filtering, anchor resolution, provenance backward traversal.
- **Concurrency** beyond the active-marker pattern (§13).

Appendix A — Delimitation from RDF

Shared intuition: there is only one sort of thing, and relations are themselves of this sort. Differences: **ownership instead of triple symmetry** (§3.2); **native reification** (§3.4); **native graded identity** as representation (§5.5); **native bitemporality** (§6.2); **no literal/resource distinction** — everything is a datum (§2.1). Conceptually closer to append-only, bitemporal models (Datomic/XTDB) than to RDF.

Appendix B — Essence in One Sentence

lakearch is an append-only substrate of exactly one entity — data that own other data as context —, which **stores, traverses and structurally matches** (type, identity and time are special contexts; identity is graded, bitemporal and federatable via content hash; anchors, materialization and an unbypassable access gate are native structures; writing is solely *append*, while finding the place lies outside; references are closed — placeholder instead of gap —, cycles permitted, every access is atomic — restructurings become jointly visible through an active marker), while **computing, judging and sorting lie deliberately outside**.

lakearch — Das Datenmodell

REGELWERK. VOLLSTÄNDIG, DOMÄNEN-FREI, TECHNOLOGIE-FREI.

Deutsche Übersetzung; maßgeblich ist die englische Fassung.

Präambel. Dieses Dokument beschreibt das Datenmodell *lakearch* erschöpfend und allgemein. Es nennt keine Domäne, keine Anwendung, keine Technologie. Es ist als Axiomensystem gefasst: wenige Grundbegriffe, daraus eindeutig folgende Regeln. Jede Regel gilt ausnahmslos; wo eine Ausnahme nötig schiene, ist die Regel

falsch gefasst.

Notation. A, B, K bezeichnen beliebige Daten; sie sind abstrakte Symbole, denn lakearch kennt keine Inhalte (§2.1, §1.4). „ $A \triangleright K$ “ liest sich „A besitzt den Kontext K“.

§1 Geltungsbereich

1.1 lakearch **speichert** Daten und ihre Kontexte.

1.2 lakearch **traversiert** entlang von Kontexten — vorwärts wie rückwärts.

1.3 lakearch **matcht strukturell**: Gleichheit auf Inhalts-/Adressebene, „zeigt ein Kontext auf ein Daten?“, Zugehörigkeit zu einer per Kontext gegebenen Menge.

1.4 lakearch **deutet nicht** (Inhalte bleiben opak, §2.1; Bedeutung entsteht erst durch das Vokabular der Ableitung, §14), **rechnet nicht** (keine Arithmetik, Ordnung, Aggregation), **wertet nicht** (erzeugt keine Konfidenz, entscheidet keine Identität, validiert keine Eingabe) und **sortiert nicht**. lakearch ist ein **passiver Speicher**: es hält Daten, ohne eigene Logik über sie auszuführen.

1.5 Alles Rechnen, Werten und Sortieren liegt in einer **Schicht über lakearch**. Sie liest per Traversierung und schreibt ihre Ergebnisse als Daten zurück.

1.6 **Zyklen sind erlaubt**. Der Graph ist kein DAG; ein Kontext darf mittelbar auf sein eigenes Besitzer-Daten zurückzeigen. Strukturelle Terminierung folgt nicht aus der Form, sondern aus der Art der Traversierung (§1.7).

1.7 Traversierung tritt in zwei Sorten auf:

- (a) **Mechanische Traversierung** — das Zugriffs-Tor (§11) und die Invalidierung (§10) — ist **deterministisch, beschränkt** (über eine Besuchsmenge) **und damit zyklensicher**. Sie matcht nur; sie rechnet und wertet nicht.
- (b) **Explorative Traversierung** — Suche, Platzierung, absichtliches Zurückgehen — liegt in der Schicht darüber (§1.5). Sie darf Zyklen nutzen und ist nicht an Determinismus gebunden.

§2 Die Entität

2.1 Es gibt genau **eine** Art von Entität: das **Daten**. Alles, was existiert, ist ein Daten.

2.2 Es gibt keine zweite Entitätsklasse — kein Schema, keine Metadaten-, Versions-, Zeit- oder Typ-Objekte als eigene Art. Diese Reduktion ist das Kernprinzip.

2.3 Ein **Bestand** ist eine zusammenhängende Menge von Daten unter genau einem Zugriffs-Tor (§11) — eine lakearch-Instanz. Alle folgenden Regeln gelten zunächst *innerhalb* eines Bestands; mehrere verbundene Bestände behandelt §12. Ein **Bereich** (§11) ist demgegenüber eine *logische* Trennung *innerhalb* eines Bestands: **Bestand $\supset$ Bereich**.

§3 Der Kontext

3.1 Ein **Kontext** ist keine eigene Entität, sondern eine **Rolle**: ein Daten K, das von einem Daten A besessen wird. K selbst ist ein gewöhnliches Daten.

3.2 Besitz ist **gerichtet**: „ $A \triangleright K$ “ ist nicht symmetrisch.

3.3 Ein **Verweis** von A auf B wird ausgedrückt, indem A einen Kontext besitzt, der auf B zielt. Die Art des Verweises (die Relation) ist selbst ein Daten.

3.4 Ein Kontext kann selbst Kontexte besitzen. **Reifikation ist kostenlos**: Aussagen über Aussagen sind nativ.

3.5 Jede Aussage des Systems ist ein Daten, das von einem Daten besessen wird. Eine andere Form von Aussage existiert nicht.

3.6 Ein Verweis zeigt **stets auf ein vorhandenes Daten**. Ein noch nicht eingetroffenes Ziel wird durch ein **Platzhalter-Daten** dargestellt — ein gewöhnliches Daten mit einem Kontext, der es als *unaufgelöst/erwartet* ausweist. Spätere Aufnahme ersetzt es (§6.3) oder verbindet sich über den Korrelations-Pfad (§5.7 b). So bleibt jeder Verweis geschlossen: es gibt keine baumelnden Verweise, nur explizit als unaufgelöst markierte Daten.

§4 Typ

4.1 **Typ** ist ein besonderer Kontext: ein Daten trägt seinen Typ als Kontext, der auf ein Typ-Daten zeigt.

4.2 Da ein Typ ein Daten ist, gibt es **keinen Meta-Typ-Regress**; Typ-Aussagen sind gewöhnliche Daten.

4.3 Daten haben keine vorgeschriebene Struktur. Struktur entsteht allein durch Kontexte. Jede Domäne ist abbildbar, **ohne das Modell zu erweitern**.

§5 Identität

5.1 Identität ist **keine intrinsische Eigenschaft**, sondern eine Aussage über Daten — und damit selbst ein Kontext.

5.2 **Speicher-Identität**: Jedes je geschriebene Daten trägt eine opake ID, vorzugsweise einen **Inhalts-Hash**. Sie adressiert; sie urteilt nicht.

5.3 **Wert-Identität**: Inhaltsgleiche Daten sind dasselbe Daten — Inhaltsadressierung dedupliziert automatisch. Ein primitives Daten existiert genau einmal.

5.4 **Referenzielle Identität** („dasselbe Ding?“) wird ausschließlich über Kontexte ausgedrückt, niemals durch destruktive Operationen.

5.5 Referenzielle Identität ist **gradiert**: eine Familie von Identitäts-Kontexten unterschiedlicher Stärke. Jeder trägt eigene Kontexte: betroffene Attribute, Konfidenz, Urheber, Zeit.

5.6 Eigenschaften einer Quelle (stabile Schlüssel, bekannte Abweichungen, Vertrauensgrad) sind Kontexte an dem Daten, das die Quelle repräsentiert.

5.7 Die Aufnahme eines Daten folgt genau einem von drei Pfaden, **erkennbar allein an Kontexten**:

(a) **Fortschreibung** — gleiche Quelle, stabiler Schlüssel, kein Widerspruch: neuer Zustand, der alte wird als ersetzt markiert.

(b) **Korrelation** — andere Quelle oder kein eindeutiger Schlüssel: immer eigenständiges Daten, zuzüglich gradierter Identitäts-Kontexte.

(c) **Konflikt** — keine Richtung trägt oder es bestehen aktive Widersprüche: als solcher modelliert; Auflösung auf die Leseseite verschoben (§8).

5.8 Welcher Pfad gilt, **wählt die schreibende Schicht (§7)** — nicht lakearch. lakearch *matcht* die Vorbedingungen (existiert ein stabiler Schlüssel? existiert ein Widerspruchs-Kontext?, §1.3); die *Wahl* und jede Wertung trifft die Schicht darüber (§1.4).

§6 Zeit

6.1 Zeit ist Daten. Zeitpunkte und Zeiträume sind Daten; Zeit-Aussagen sind besondere Kontexte.

6.2 Es gibt **zwei Zeitachsen**: **Aufzeichnungszeit** (wann das System es erfuhr) und **Gültigkeitszeit** (wann der Sachverhalt in der Welt gilt). Sie dürfen auseinanderfallen.

6.3 Das Modell ist **append-only**: neues Wissen kommt als neue Daten hinzu. Ein **Ersetzungs-Kontext** markiert Älteres als überholt, ohne es zu löschen.

6.4 Eine „Version“ ist nichts Gespeichertes, sondern eine **Leseregel** (§8).

§7 Das Schreiben

7.1 Die **einzige Mutation** ist **append**: genau ein neues Daten samt seinen Kontexten kommt hinzu. Es wird **nie geändert und nie gelöscht**. Ersetzung (§6.3) und der Aktiv-Marker (§13) sind die *einzigsten* Formen von „Änderung“ — beide selbst nur weitere Schreibvorgänge.

7.2 **Was** geschrieben wird und **wohin** es sich per Kontext bindet, entscheidet die **Schicht über lakearch** (§1.5): sie traversiert zum Orientieren (§1.7 b), matcht zum Prüfen (§1.3) und schreibt dann **einmal**. lakearch *findet den Platz nicht und beurteilt die Verbindung nicht*; es nimmt das fertige Ergebnis auf.

7.3 **Append rechnet und wertet nicht** (§1.4). Auch die Entscheidungskriterien der schreibenden Schicht liegen als Daten im Bestand; sie zu *lesen* ist Traversierung, sie *anzuwenden* ist Sache der Schicht darüber. Daraus folgt der Leitsatz: *aller Zustand liegt in lakearch, alle Berechnung darüber — auch die Berechnung, die entscheidet, wohin neuer Zustand kommt*.

7.4 Betrifft ein Schreibvorgang mehrere Daten, ist sein Ergebnis erst wirksam, wenn der Aktiv-Marker es freigibt (§13); bis dahin gelten die Teile als inaktiv.

§8 Auflösung beim Lesen

8.1 Was gradierte Identität (§5), Zeit (§6) und Konflikte bedeuten, wird **beim Lesen** entschieden, nicht beim Schreiben.

8.2 Eine Abfrage wählt implizit: Konfidenz-Schwellen, Quell-Gewichtung, Zeitpunkt und Zeitachse.

8.3 Dieselbe Frage an denselben Bestand kann je nach Wahl verschiedene Ergebnisse liefern. Mehrdeutigkeit wird beim Lesen ehrlich behandelt, nicht beim Schreiben verborgen.

8.4 Lesen erzeugt nichts im Speicher; es **projiziert** aus dem vorhandenen Bestand. Historische Zustände werden so rekonstruiert.

§9 Strukturen der Identitäts-Auflösung

lakearch hält die Strukturen; das Auflösen selbst (Schwellen, Entscheidungen) geschieht in der Schicht darüber (§1.5).

9.1 **Repräsentantensystem**: Tragen mehrere Daten dieselbe referenzielle Identität, existiert ein eigenes **Anker-Daten** (die Klasse). Die Repräsentanten verweisen per Mitgliedschafts-Kontext auf den Anker.

9.2 Anderes verweist auf den **Anker**, nicht auf einen einzelnen Repräsentanten. Ein Repräsentant wird **niemals** in den Anker umgewandelt (§6.3).

9.3 Mitgliedschaft ist **gradiert und revidierbar** — ein Kontext, kein destruktiver Zusammenschluss.

9.4 Ein **Split** entsteht durch neue Kontexte, die alte ersetzen; betroffene Repräsentanten werden re-verwiesen.

9.5 **Kuratierung** fügt ausschließlich reversible Kontexte hinzu (*verbergen, ersetzen, Mitgliedschaft*); die Leseseite filtert sie. Es wird nie gelöscht; physisches Entfernen ist Compaction (§15).

§10 Materialisierung

10.1 Ein berechnetes Ergebnis darf als Daten gespeichert werden; ein neueres ersetzt das ältere (§6.3).

10.2 Ein berechnetes Daten trägt seine **Herkunft als Kontext** — die Bindung an die Eingaben, aus denen es entstand.

10.3 Ändert sich eine Eingabe (auch rückwirkend, §6), werden die betroffenen Ergebnisse durch **Rückwärts-Traversierung** der Herkunfts-Kontexte gefunden (Invalidierung; mechanisch, §1.7 a). Das Neu-Berechnen liegt außerhalb (§1.5).

§11 Zugriff

11.1 **Beschreibung.** Ein **Bereich** ist ein Daten; Zugehörigkeit ist ein Kontext (ein Daten kann mehreren Bereichen angehören). Ein Bereich trennt *logisch innerhalb* eines Bestands (§2.3). Eine **Berechtigung** ist ein Daten mit Kontexten *Subjekt, Bereich, Recht, Zeit, Urheber* — auditierbar und bitemporal.

11.2 **Durchsetzung.** Jeder Lesevorgang passiert ein **unumgebares Tor**. Das Tor bildet die Sichtbarkeit aus den Berechtigungen (reines strukturelles Matching: *Bereich* ∈ *gewährte Bereiche*, §1.3; mechanisch, §1.7 a) und wendet sie **vor jeder Auflösung** an.

11.3 **Filtern vor Auflösen** ist zwingend: nicht-sichtbare Daten werden entfernt, bevor referenzielle Identität (§9) aufgelöst wird — damit nichts Nicht-Sichtbares in ein erlaubtes Ergebnis einfließt.

11.4 **Entzug** ist ein neuer Berechtigungs-Kontext, der den alten ersetzt; künftige Lesevorgänge filtern ihn, bereits Gelesenes bleibt (§6.3).

11.5 Das Tor ist Teil von lakearch — *immer aufgerufen, manipulationssicher, minimal*. Es matcht nur (§1.3); es rechnet nicht.

§12 Föderation

12.1 Ein einzelner Bestand **und** mehrere verbundene Bestände sind beides möglich — mit denselben Regeln.

12.2 **Innerhalb eines Bestands** trennen Bereiche (§11) logisch.

12.3 **Über Bestände hinweg** tragen inhaltsgleiche Daten denselben Inhalts-Hash (§5.2) und sind damit bestand-übergreifend identisch — automatischer Abgleich. Referenzielle Identität wird über den Korrelations-Pfad (§5.7 b) verbunden; einen Bestand in einen anderen aufzunehmen *ist* dieser Pfad.

12.4 Inhalts-IDs sind bestand-unabhängig; **Anker-IDs (§9.1) sind bestand-lokal** und werden beim Zusammenführen über gradierte Identität versöhnt.

§13 Atomarität

13.1 Jeder Datenzugriff — **lesend wie schreibend** — ist **atomar**: unteilbar und ohne beobachtbaren Zwischenzustand. Kein Zugriff beobachtet je einen halb-vollzogenen Schreibvorgang; er sieht den Bestand entweder vollständig davor oder vollständig danach.

13.2 Für den **einzelnen Schreibvorgang** ist Atomarität strukturell gegeben: die einzige Mutation ist *append* (§7.1), und ein Daten wird nie geändert und nie gelöscht. Das neue Daten samt Kontexten wird als Ganzes sichtbar oder gar nicht; ein teil-geschriebenes Daten ist unbeobachtbar.

13.3 Für das **einzelne Lesen** folgt Atomarität aus der Unveränderlichkeit: Lesen projiziert (§8.4) aus bereits vorhandenen, unveränderlichen Daten und beobachtet keinen von einem nebenläufigen Schreiben erzeugten Zwischenzustand.

13.4 Für das **Lesen über mehrere Daten** — Traversierung, Abfrage — folgt Atomarität aus dem impliziten **Zeitschnitt** (§8.2) zusammen mit der Append-only-Unveränderlichkeit (§7.1): Der Lesevorgang projiziert (§8.4) gegen genau einen Zeitpunkt. Da Schreiben allein *append* ist und Bestehendes nie ändert, wächst der Bestand nur; ein nebenläufig angehängtes Daten liegt hinter dem Schnitt und bleibt unbeobachtet. So sieht

auch der zusammengesetzte Lesevorgang den Bestand vollständig vor oder vollständig nach einem nebenläufigen Schreiben (§13.1) — nie einen Mischzustand.

13.5 Für den **mehrere Daten betreffenden Umbau** (Zusammenführen/Spalten §9, Berechtigungs-Wechsel §11) entsteht gemeinsame Sichtbarkeit durch ein einziges abschließendes **Aktiv-Schreiben**. Bis der Aktiv-Marker gesetzt ist, gelten die Teile als inaktiv; Traversierung ignoriert sie (§7.4).

13.6 Ein halb-vollzogener Umbau ist damit unsichtbar, bis er vollständig ist — **Atomarität ohne Transaktions-Maschinerie**. Nebenläufigkeit über dieses Muster hinaus ist Umsetzungssache (§15).

§14 Ableitung & Trennung

14.1 Eine **domänenspezifische Ableitung** erweitert das Modell nicht. Sie ist Daten *innerhalb* eines lakearch-Bestands: Vokabular (Typ- und Relations-Daten), Konventionen, Konfiguration.

14.2 **Trennungsregel**. Vokabular gehört in die Ableitung. Ein neues Speicher-, Traversier- oder Match-Primitiv gehört in lakearch und muss domänen-frei sein. Rechnen, Werten und Sortieren gehört in die Schicht darüber (§1).

14.3 lakearch kennt **keine** Ableitung namentlich. Es stellt nur Primitive bereit, in denen sich jede Ableitung vollständig ausdrückt.

§15 Bewusst offen (Implementierung & Betrieb)

Diese Punkte sind kein Modell-Bestandteil, sondern Umsetzungs-Entscheidungen:

- **ID-Schema** (Inhalts-Hash / UUID / hybrid), **Serialisierung**, **Abfrage- und Traversier-Sprache**.
- **Compaction**: wann Überholtes oder Verborgenes *physisch* entfernt werden darf (Spannung Append-only ↔ Lösch-Pflichten).
- **Indexierung & Leistung** von Bereichs-Filter, Anker-Auflösung, Herkunfts-Rückwärts-Traversierung.
- **Nebenläufigkeit** über das Aktiv-Marker-Muster (§13) hinaus.

Anhang A — Abgrenzung zu RDF

Geteilte Intuition: es gibt nur eine Sorte Ding, und Beziehungen sind selbst von dieser Sorte. Unterschiede: **Besitz statt Tripel-Symmetrie** (§3.2); **native Reifikation** (§3.4); **native gradierte Identität** als Repräsentation (§5.5); **native Bitemporalität** (§6.2); **keine Literal/Resource-Unterscheidung** — alles ist Daten (§2.1). Konzeptionell näher an append-only, bitemporalen Modellen (Datomic/XTDB) als an RDF.

Anhang B — Wesen in einem Satz

lakearch ist ein append-only Substrat aus genau einer Entität — Daten, die andere Daten als Kontext besitzen —, das **speichert, traversiert und strukturell matcht** (Typ, Identität und Zeit sind besondere Kontexte; Identität ist gradiert, bitemporal und über Inhalts-Hash förderierbar; Anker, Materialisierung und ein unumgebares Zugriffs-Tor sind native Strukturen; Schreiben ist allein *append*, während das Finden des Platzes außerhalb liegt; Verweise sind geschlossen — Platzhalter statt Lücke —, Zyklen erlaubt, jeder Zugriff ist atomar — Umbauten werden durch einen Aktiv-Marker gemeinsam sichtbar), während **Rechnen, Werten und Sortieren bewusst außerhalb** liegen.